

KBR PDF Service — Guia de Uso

Serviço genérico de geração e processamento de PDF com branding KINTO. Oferece duas capacidades principais:

1. **Geração de relatórios PDF** — PDFs estruturados com cover, seções, branding KINTO
2. **Processamento de imagens e PDFs** — compressão de imagens (sharp), merge de PDFs (pdf-lib), rasterização (mupdf)

Visual:

- **Cores:** paleta KINTO (brand blue `#00708D`, neutras, semânticas)
 - **Header:** logo `KINTO_SQ_BLUE` + título configurável + referência
 - **Cover:** logo `KINTO_BLUE` em destaque + kicker + título + metadata + summary + overview cards
 - **Footer:** texto confidencial + paginação automática
 - **Fontes:** família Inter (Regular, Light, Medium, SemiBold, Bold)
-

Instalação

O service depende de:

- `pdfkit` (`>=0.18`) — geração de PDF
- `svg-to-pdfkit` (`>=0.1.8`) — renderização de SVG inline
- `sharp` (`>=0.35`) — compressão de imagens
- `pdf-lib` (`>=1.17`) — merge de PDFs
- `mupdf` (`>=1.27`) — rasterização de PDF em JPEG (WASM)
- Pasta `assets/` com `fonts/` (Inter TTFs) e `svg/` (logos KINTO)
- `gs` (Ghostscript) no PATH para lidar com PDFs criptografados (opcional)

Essas dependências já estão no `package.json` do service.

PROF

Import

```
// Geração de relatório PDF com branding KINTO
import { generatePdf } from './services/kbr-pdf-service/index.js';
import type {
  PdfReportInput,
  PdfReportConfig,
  PdfSection,
  PdfSectionItem,
  PdfMetadataField,
  PdfOverviewCard,
} from './services/kbr-pdf-service/index.js';

// Processamento de imagens e PDFs
```

```
import {
  imageCompressionService,
  ImageCompressionService,
  PdfMergeService,
  pdfAttachmentService,
  PdfAttachmentService,
  PdfIntegrationService,
  rasterizePdfToJpegs,
} from './services/kbr-pdf-service/index.js';
import type {
  CompressionOptions,
  CompressionResult,
  PdfIntegrationOptions,
} from './services/kbr-pdf-service/index.js';
```

API

```
async function generatePdf(input: PdfReportInput): Promise<Buffer>
```

Retorna um **Buffer** contendo o PDF completo, pronto para:

- Gravar em disco (`fs.writeFileSync('output.pdf', buffer)`)
- Enviar como attachment por e-mail (SES)
- Upload para S3

Lança erro se **input** for nulo ou não contiver `summary` nem `sections`.

Tipos Principais

PdfReportInput

```
interface PdfReportInput {
  config?: PdfReportConfig;
  metadata?: PdfMetadataField[];
  summary?: string;
  summaryTitle?: string;
  sections?: PdfSection[];
  overviewCards?: PdfOverviewCard[];
}
```

PdfReportConfig

```
interface PdfReportConfig {
  headerTitle?: string; // Default: "KINTO Report"
  reference?: string; // Default: auto-derivado do metadata
  coverKicker?: string; // Default: "TECHNICAL REPORT"
  coverTitle?: string; // Default: "Report"
  footerText?: string; // Default: "© KINTO MOBILITY · DOCUMENTO
CONFIDENCIAL"
}
```

PdfSection

```
interface PdfSection {
  title: string;
  descriptor?: string;
  items: PdfSectionItem[];
}
```

PdfSectionItem

```
interface PdfSectionItem {
  title: string;
  description: string;
  suggestion?: string;
  file?: string;
  line?: number;
  severity?: 'high' | 'medium' | 'low' | string;
}
```

Exemplos de Uso

Exemplo 1: Relatório de Code Review

```
import { generatePdf } from './services/kbr-pdf-service/index.js';
import { writeFileSync } from 'fs';

const buffer = await generatePdf({
  config: {
    headerTitle: 'KINTO Code Quality Report',
    coverKicker: 'TECHNICAL AUDIT',
    coverTitle: 'Code Analysis Report',
    footerText: '© KINTO MOBILITY · AUDITORIA TÉCNICA CONFIDENCIAL',
  },
  metadata: [
    { label: 'Project', value: 'kbr-checklist-entrega' },
```

```

    { label: 'Repository', value: 'kinto/kbr-checklist-entrega' },
    { label: 'PR', value: '#42', link: 'https://github.com/kinto/kbr-checklist-entrega/pull/42' },
    { label: 'Commit', value: 'a1b2c3d4e5f6' },
    { label: 'Date', value: '06/26/2026' },
  ],
  summary: 'Foram identificados 3 riscos de severidade alta relacionados a injeção de SQL e uso de eval. Recomenda-se correção imediata antes do merge.',
  overviewCards: [
    { label: 'Risks', value: 3 },
    { label: 'Error Handling', value: 2 },
    { label: 'Performance', value: 1 },
    { label: 'Best Practices', value: 4 },
  ],
  sections: [
    {
      title: 'Risks',
      descriptor: 'Findings that may compromise security, reliability, or correctness.',
      items: [
        {
          title: 'SQL Injection em query dinâmica',
          description: 'A query em getUser() concatena input do usuário diretamente na string SQL sem sanitização.',
          suggestion: 'Usar parameterized queries ou prepared statements.',
          file: 'src/repositories/user-repo.ts',
          line: 45,
          severity: 'high',
        },
        {
          title: 'Uso de eval() para parsing',
          description: 'eval() é utilizado para parsear JSON dinâmico, permitindo execução arbitrária de código.',
          suggestion: 'Substituir por JSON.parse() com validação de schema.',
          file: 'src/utils/parser.ts',
          line: 12,
          severity: 'high',
        },
      ],
    },
    {
      title: 'Performance',
      descriptor: 'Optimization opportunities and costly patterns.',
      items: [
        {
          title: 'Loop O(n²) na detecção de duplicatas',
          description: 'Array.includes() dentro de for loop resulta em complexidade quadrática.',
          suggestion: 'Usar Set para lookup O(1).',
          file: 'src/services/dedup.ts',
        },
      ],
    },
  ],

```

```

        line: 78,
        severity: 'medium',
      },
    ],
  },
],
});

writeFileSync('code-review-report.pdf', buffer);

```

Exemplo 2: Relatório de Checklist de Entrega

```

const buffer = await generatePdf({
  config: {
    headerTitle: 'KINTO Checklist Report',
    coverKicker: 'DELIVERY CHECKLIST',
    coverTitle: 'Vehicle Inspection Report',
  },
  metadata: [
    { label: 'Placa', value: 'ABC-1234' },
    { label: 'Modelo', value: 'Toyota Corolla 2024' },
    { label: 'Inspetor', value: 'João Silva' },
    { label: 'Data', value: '26/06/2026' },
  ],
  summary: 'Veículo inspecionado com 2 não conformidades identificadas no exterior. Demais itens em conformidade.',
  summaryTitle: 'Resumo da Inspeção',
  overviewCards: [
    { label: 'Conforme', value: 45 },
    { label: 'Não Conforme', value: 2 },
    { label: 'N/A', value: 3 },
  ],
  sections: [
    {
      title: 'Não Conformidades',
      descriptor: 'Itens que requerem ação corretiva antes da entrega.',
      items: [
        {
          title: 'Risco no para-brisa dianteiro',
          description: 'Risco de aproximadamente 15cm na região inferior do para-brisa, lado do motorista.',
          suggestion: 'Substituição do para-brisa necessária.',
          severity: 'high',
        },
        {
          title: 'Amassado na porta traseira direita',
          description: 'Amassado de ~5cm de diâmetro na região central da porta.',
          suggestion: 'Encaminhar para funilaria – reparo estimado em 2 dias.',

```

```

        severity: 'medium',
      },
    ],
  },
],
});

```

Exemplo 3: Relatório Semanal

```

const buffer = await generatePdf({
  config: {
    headerTitle: 'KINTO Weekly Report',
    coverKicker: 'WEEKLY SUMMARY',
    coverTitle: 'Fleet Operations Report',
    footerText: '© KINTO MOBILITY · USO INTERNO',
  },
  metadata: [
    { label: 'Período', value: '17/06/2026 – 23/06/2026' },
    { label: 'Região', value: 'São Paulo - SP' },
    { label: 'Gerado por', value: 'Sistema Automatizado' },
  ],
  summary: 'Semana com operação estável. 142 entregas realizadas, 3 sinistros registrados (todos de baixa severidade). SLA de entrega mantido em 98.6%',
  summaryTitle: 'Visão Geral da Semana',
  overviewCards: [
    { label: 'Entregas', value: 142 },
    { label: 'Sinistros', value: 3 },
    { label: 'SLA', value: '98.6%' },
    { label: 'Veículos Ativos', value: 87 },
  ],
  sections: [
    {
      title: 'Sinistros',
      descriptor: 'Ocorrências registradas no período.',
      items: [
        {
          title: 'Colisão leve – Corolla ABC-1234',
          description: 'Colisão traseira em baixa velocidade. Danos estéticos no para-choque traseiro.',
          severity: 'low',
        },
        {
          title: 'Vidro quebrado – Yaris DEF-5678',
          description: 'Vidro lateral traseiro direito quebrado por tentativa de furto.',
          severity: 'low',
        },
        {
          title: 'Pneu furado – HB20 GHI-9012',

```

```

        description: 'Pneu dianteiro esquerdo furado em via urbana.
Substituído pelo estepe.',
        severity: 'low',
    },
],
},
{
    title: 'Destques Operacionais',
    items: [
        {
            title: 'Recorde de entregas no dia 19/06',
            description: '32 entregas realizadas em um único dia,
superando a média de 24.',
        },
        {
            title: 'Nova rota otimizada zona leste',
            description: 'Implementação da rota alternativa reduziu tempo
médio de entrega em 12 minutos.',
        },
    ],
},
],
});

```

Exemplo 4: Relatório Simples (só summary)

```

const buffer = await generatePdf({
  config: {
    coverTitle: 'Status Report',
    coverKicker: 'MONTHLY UPDATE',
  },
  metadata: [
    { label: 'Month', value: 'June 2026' },
    { label: 'Author', value: 'Sandro' },
  ],
  summary: 'All systems operational. No incidents reported.
Infrastructure costs within budget at 94% of allocated spend.',
});

```

PROF

Assets Necessários

O service espera encontrar os assets em **assets/** (relativo ao CWD ou ao source file):

```

assets/
├─ fonts/
│   └─ Inter-Regular.ttf
│   └─ Inter-Light.ttf

```

```
|   |— Inter-Medium.ttf
|   |— Inter-SemiBold.ttf
|   |— Inter-Bold.ttf
|— svg/
|   |— KINTO_BLUE.svg      ← Logo grande na capa
|   |— KINTO_SQ_BLUE.svg  ← Logo quadrado no header
|   |— KINTO_SQ.svg
|   |— KINTO_WHITE.svg
|   |— KINTO_SHARE_WHITE.svg
```

Para usar em outra Lambda

1. Copie a pasta `services/kbr-pdf-service/` para o diretório da Lambda
2. Copie a pasta `assets/` (fonts + svg) para o root da Lambda
3. Garanta as dependências no `package.json`:
 - `pdfkit`, `svg-to-pdfkit` — para geração de relatório
 - `sharp` — para compressão de imagens
 - `pdf-lib` — para merge de PDFs
 - `mupdf` — para rasterização de PDFs em JPEG
4. Adicione o type declaration `svg-to-pdfkit.d.ts` ao projeto (ou declare module)
5. Import e use `generatePdf()` e/ou os serviços de processamento

Processamento de Imagens e PDFs

Módulos extraídos do checklist de sinistro-casco, projetados para manter o PDF final leve e manejar documentos anexos (boletins, orçamentos, CNH, etc.).

ImageCompressionService

Comprime imagens com `sharp` para otimizar o tamanho final do PDF.

Configuração padrão:

Parâmetro	Valor
maxWidth	1200 px
maxHeight	1600 px
jpegQuality	80 (mozjpeg)
pngCompressionLevel	8
targetMaxSizeKb	500 KB

```
import { imageCompressionService } from './services/kbr-pdf-  
service/index.js';
```



```
import type { CompressionOptions, CompressionResult } from
'./services/kbr-pdf-service/index.js';

// Comprimir uma única imagem
const result: CompressionResult = await
imageCompressionService.compressImage(
  imageBuffer,          // Buffer da imagem original
  'image/jpeg',         // MIME type
  { quality: 75 }       // Opções (opcional)
);
console.log(`Redução: ${result.compressionRatio.toFixed(1)}%`);
// result.buffer contém a imagem comprimida

// Comprimir lote de imagens em paralelo
const batch = await imageCompressionService.compressBatch([
  { buffer: img1, contentType: 'image/jpeg', nome: 'foto_frontal.jpg' },
  { buffer: img2, contentType: 'image/png', nome: 'cnh.png' },
  { buffer: img3, contentType: 'image/jpeg', nome: 'dano_lateral.jpg' },
]);
// Cada item: { buffer, contentType, nome, compressionInfo }

// Verificar se compressão é necessária
if (imageCompressionService.shouldCompress(imageBuffer)) {
  // imagem > 500 KB, comprimir
}

// Estimar tamanho final do PDF
const estimatedSize = imageCompressionService.estimatePdfSize(images);
```

CompressionOptions:

```
interface CompressionOptions {
  maxWidth?: number;          // Default: 1200
  maxHeight?: number;         // Default: 1600
  quality?: number;           // Default: 80
  format?: 'jpeg' | 'png' | 'webp' | 'auto'; // Default: 'auto'
  (detecta pelo contentType)
}
```

PdfAttachmentService / rasterizePdfToJpegs

Rasteriza PDFs em imagens JPEG e embute num documento PDFKit. Cada página vira uma imagem (120 DPI, Q75).

Lida automaticamente com PDFs criptografados:

1. Tenta abrir com mupdf diretamente
2. Se falhar (criptografia) → normaliza via Ghostscript (gs) → tenta novamente

3. Se ambos falharem → lança erro

```
import { pdfAttachmentService, rasterizePdfToJpegs } from
  './services/kbr-pdf-service/index.js';

// Rasterizar PDF (possivelmente criptografado) em array de JPEGs
const jpegPages: Buffer[] = await rasterizePdfToJpegs(pdfBuffer);
// Array de Buffers JPEG (120 DPI, Q75), um por página
// Funciona mesmo com PDFs criptografados (Ghostscript normaliza)

// Embutir PDF como imagens rasterizadas num documento PDFKit
await pdfAttachmentService.includePdfAsImages(
  pdfkitDoc,
  boletimBuffer,
  'Boletim de Ocorrência' // título exibido no topo de cada página
);

// Verificar se um buffer é PDF válido (magic bytes)
const info = pdfAttachmentService.analyzeBasicPdfInfo(buffer);
// { sizeKB: 450, isPdf: true, hasValidHeader: true }
```

Requisitos:

- **mupdf** — rasterização via WASM
- **gs** (Ghostscript) no PATH — apenas para PDFs criptografados (opcional)

Fluxo Completo — Checklist com Fotos e PDFs

Todos os campos de upload (CNH, placa, boletim, dano) podem receber imagem OU PDF.
O PDF é rasterizado e tratado como imagem — aparece inline na seção correspondente.

PROF

```
import {
  imageCompressionService,
  rasterizePdfToJpegs,
  pdfAttachmentService,
} from './services/kbr-pdf-service/index.js';

/**
 * Verifica se o buffer é um PDF (magic bytes %PDF-)
 */
function isPdf(buffer: Buffer): boolean {
  return pdfAttachmentService.analyzeBasicPdfInfo(buffer).isPdf;
}

/**
 * Normaliza um arquivo de upload para imagem JPEG, independente do
 * formato original.
 * - Imagem (PNG/JPEG/WebP) → comprime com sharp
```

```

* - PDF (normal ou criptografado) → rasteriza cada página em JPEG
*/
async function normalizeFileToImages(
  buffer: Buffer,
  contentType: string
): Promise<Buffer[]> {
  if (isPdf(buffer)) {
    // PDF → rasteriza (lida com criptografia automaticamente)
    return await rasterizePdfToJpegs(buffer);
  }

  // Imagem → comprime
  const result = await imageCompressionService.compressImage(buffer,
contentType);
  return [result.buffer];
}

// Uso no processamento do checklist:
for (const section of formData.sections) {
  for (const file of section.files) {
    const images = await normalizeFileToImages(file.buffer,
file.contentType);
    // images = array de Buffers JPEG prontos pra renderizar no PDFKit
    // Se era imagem: 1 item. Se era PDF de 3 páginas: 3 itens.
  }
}

```